SecureStay: Project Documentation and 2-Day Execution Plan

Word Document for Cloud Computing Project Submission

Important note: This document is intended to be submitted as a Microsoft Word document (.docx). It includes the required sections—Introduction, Architecture, Implementation Steps, Challenges Faced, and Lessons Learned—and also adds a detailed 2-day team execution plan so that each team member clearly understands what to do, what to build, and what to complete.

1. Introduction

SecureStay is a cloud-native hotel booking system designed to demonstrate core cloud computing principles such as scalability, high availability, security, containerized deployment, and automated delivery. The system models a realistic booking platform where users can search hotels, reserve rooms, complete a payment workflow, and receive notifications after important events. The solution uses a microservices architecture because cloud-native systems benefit from modular design, stateless services, and independent scaling.

This project also places strong emphasis on cloud security. The design includes JWT-based authentication, role-based access control, encrypted communication, protected secrets, and a PCI-DSS-oriented payment approach where raw card data is not stored. These decisions match the course focus on secure and resilient cloud applications.

No core functionality is reduced in the final project scope. The team will still implement all major business functions expected from the hotel booking system: user authentication, hotel search, room booking, payment handling, and notification delivery. The main strategy is not to remove functionality, but to keep the user interface simple so more time can be invested in architecture, security, deployment, and integration.

2. Architecture

SecureStay uses a microservices architecture with an API Gateway as the single entry point. Client requests first reach the gateway, where routing, JWT validation, and basic rate control are handled. The gateway then forwards requests to the relevant backend service. PostgreSQL is used as the relational database because booking and payment operations require consistency and transactional reliability. RabbitMQ is used for asynchronous messaging so that background operations such as notifications do not block user-facing requests.

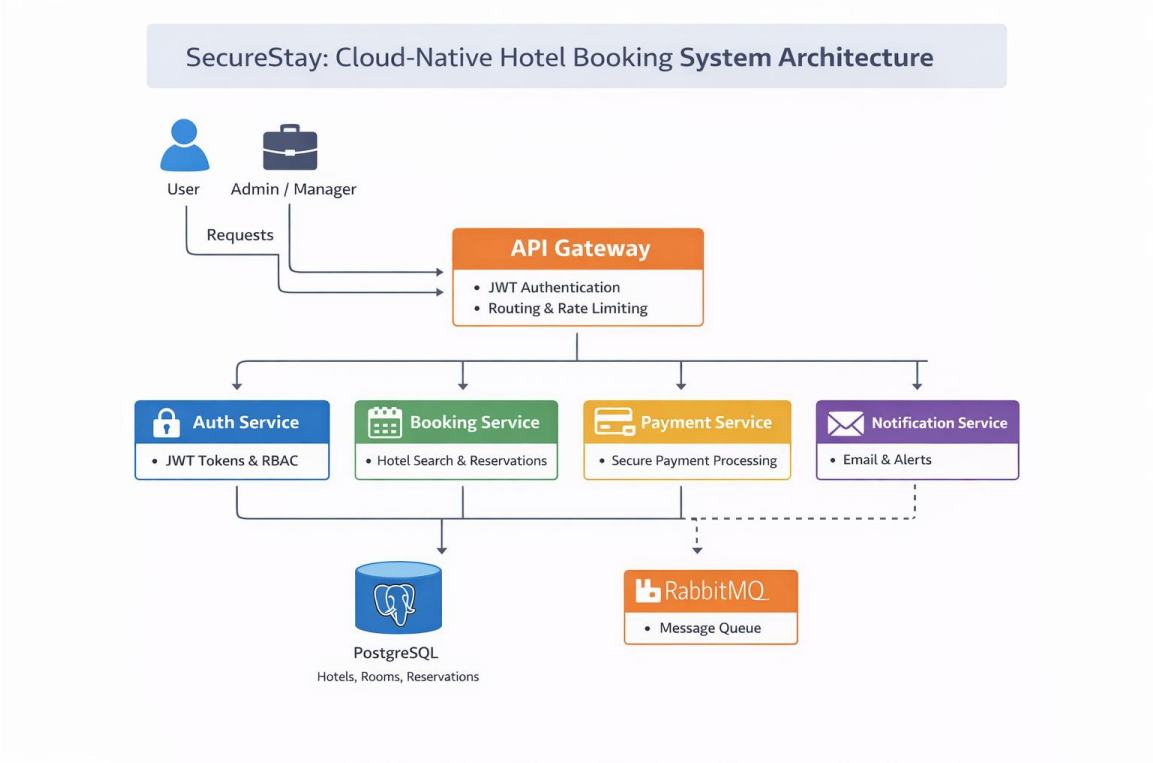

Figure 1. Final SecureStay cloud-native architecture

Component	Responsibility
API Gateway	Routes requests, validates JWT tokens, and acts as the single public entry point.
Auth Service	Handles registration, login, token issuance, and role-based access control.
Booking Service	Manages hotel search, room availability, reservations, and booking records.
Payment Service	Processes payments securely and stores transaction status without saving raw card data.
Notification Service	Consumes events from RabbitMQ and sends booking or payment notifications.
PostgreSQL	Stores structured entities such as users, hotels, rooms, bookings, and payments.
RabbitMQ	Supports asynchronous event flow between booking, payment, and notification services.
Docker + Kubernetes	Provide containerization, service deployment, self-healing, and scaling.

3. Implementation Steps

Step 1 – Finalize scope and architecture

Confirm the complete feature set, service boundaries, database entities, API contracts, and event flow. Freeze the architecture early so the team can work in parallel without confusion.

Step 2 – Prepare repositories and project structure

Create the GitHub repository, define service folders, prepare branch naming, and agree on coding standards. This step avoids integration problems later.

Step 3 – Design database and contracts

Create PostgreSQL schema for users, hotels, rooms, bookings, and payments. Define REST endpoints and RabbitMQ message formats before full coding starts.

Step 4 – Develop the microservices

Implement Auth, Booking, Payment, and Notification services with clear responsibilities and internal testing.

Step 5 – Add security controls

Integrate JWT authentication, RBAC, request validation, secret handling, and payment-safe design.

Step 6 – Containerize and deploy

Create Dockerfiles and Kubernetes manifests for each service. Configure service exposure, environment variables, readiness checks, and replicas.

Step 7 – Integrate, test, and prepare demo

Run end-to-end tests for login, hotel search, booking, payment, and notification. Fix integration issues and finalize demo script, report, and video flow.

4. Challenges Faced

Distributed system complexity

A microservices approach improves modularity, but integration becomes more difficult because services must communicate correctly through APIs and messaging.

Transaction coordination

A hotel booking action can involve booking records, payment status, and notifications. Maintaining consistent system state across multiple services requires careful event design.

Security overhead

Authentication, encryption, secret management, and input validation improve safety but add configuration and testing effort.

Deployment and infrastructure learning curve

Docker, Kubernetes, and CI/CD provide strong cloud value, but they also introduce setup complexity, especially under tight deadlines.

Time pressure

Completing a cloud-native project in two days is only possible if tasks are clearly divided, features are integrated in parallel, and every member focuses on defined outputs.

5. Lessons Learned

The project shows that cloud-native design is not only about writing backend code. It is equally about secure architecture, service separation, event-driven communication, deployment automation, and operational reliability. The team also learns the shared responsibility model of cloud security, where the cloud provider secures the underlying infrastructure while developers secure applications, APIs, secrets, and data handling. In addition, the project highlights the value of declarative deployment through Kubernetes and Infrastructure as Code, because repeatable deployment reduces configuration drift and improves recovery.

6. Team Work Division for 2 Days

The team has four members. To finish within two days without reducing core functionality, the work must be divided by architectural responsibility and executed in parallel. Each member must have clear deliverables, a completion target, and required integration points.

Member	Main Responsibility	What this member must build	Final output by end of Day 2
Member 1	Auth + Security	User registration/login APIs, JWT generation and validation logic, role setup (customer/admin), input validation, secret loading, security notes for report.	Working Auth Service, test tokens, RBAC rules, and security section content.
Member 2	Booking + Database	PostgreSQL schema, seed data for hotels/rooms, hotel search API, room availability check, reservation creation, booking retrieval API.	Working Booking Service with database connected and sample data ready for demo.
Member 3	Payment + Notification + RabbitMQ	Payment API, payment status storage, publish events to RabbitMQ, consume events in Notification Service, email or simulated notification output.	Working Payment and Notification services with queue-based event flow.
Member 4	Gateway + DevOps + Integration	API Gateway routes, Dockerfiles, docker-compose or local integration support, Kubernetes YAML, GitHub Actions pipeline, deployment commands, final integration support.	Gateway, containers, K8s manifests, CI/CD workflow, and deployment guide.

Clear 2-Day Achievement Plan

The following plan is written so that each member can read it and immediately understand what to do, what to finish, and how their work connects with the rest of the team.

Day 1 – Build the functional core

Member 1 – Auth and Security Lead

- Create the Auth Service project and define endpoints such as /register, /login, and /validate-token.
- Implement user model and password hashing. If time is limited, seed one admin user and one normal user in the database.
- Generate JWT tokens after successful login and include user role claims such as customer or admin.
- Share the token format and expected Authorization header with all other members so integration is consistent.
- Prepare a short security checklist: input validation, no hard-coded credentials, secure environment variables, and protected admin routes.

Member 2 – Booking and Database Lead

- Design the database schema for hotels, rooms, bookings, users, and payments. Create SQL scripts or migration files.
- Insert sample hotels and rooms because the demo must show hotel search and room reservation clearly.
- Build endpoints such as /hotels, /rooms/search, /bookings/create, and /bookings/:id.
- Add room availability logic so the system can reject already booked rooms or unavailable dates.
- Share database connection details and schema documentation with other members early on.

Member 3 – Payment, Queue, and Notification Lead

- Create a Payment Service with an endpoint such as /payments/process that accepts a booking reference and simulated payment data.
- Store payment result as success or failure without storing raw card details.
- Publish payment success and booking success events to RabbitMQ.
- Create a Notification Service consumer that reads queue messages and produces email output or console notification messages.
- Ensure the notification message includes useful information such as booking ID, hotel name, and payment status for demo purposes.

Member 4 – Gateway, Containers, and Deployment Lead

- Build the API Gateway and define routes that forward requests to Auth, Booking, Payment, and Notification services.
- Add token verification middleware at the gateway for protected routes.
- Write Dockerfiles for all services and check that each service starts independently.
- Prepare Kubernetes Deployment and Service YAML files with environment variables and replicas.
- Set up the GitHub Actions workflow or, at minimum, prepare a working CI/CD draft that builds Docker images automatically.

Day 1 integration checkpoint

At the end of Day 1, the team must verify that login, hotel search, booking creation, payment, and notification are already working together. If integration is delayed, Day 2 will become difficult.

Day 2 – Integration, deployment, testing, and documentation

Member 1 – Day 2 goals

- Test protected routes through the API Gateway and confirm unauthorized requests are blocked.
- Validate role behavior so that customer and admin operations are separated properly.
- Write the security-related paragraphs for the final document and provide screenshots or API proof if needed.
- Help the team verify that secrets are loaded from environment variables rather than being hard-coded.

Member 2 – Day 2 goals

- Fix any booking or schema issues found during integration.
- Prepare seed data that makes the demo easy to understand, with at least a few hotels and multiple room types.
- Run booking test cases: successful booking, invalid room selection, and retrieval of created booking records.
- Provide database schema description for the README and document.

Member 3 – Day 2 goals

- Connect booking and payment flow so a successful booking or payment event triggers a notification.
- Improve queue reliability by checking message payload format and consumer handling.
- Prepare demo evidence showing asynchronous processing, such as logs or console output after payment completion.
- Write the communication-method explanation for the report: REST for synchronous requests and RabbitMQ for asynchronous events.

Member 4 – Day 2 goals

- Containerize the full system and verify all containers run together.
- Apply Kubernetes manifests or local cluster deployment and confirm service connectivity.
- Test basic scaling by running more than one replica for at least one core service.
- Finalize deployment commands, CI/CD workflow, architecture diagram placement, and the execution steps in the report.

Final team acceptance checklist

- User can log in and receive a JWT token.
- User can search hotels and view available rooms.
- User can create a booking record in PostgreSQL.
- Payment service processes a simulated payment and stores the result.
- RabbitMQ transfers an event to the Notification Service.
- Notification is produced after booking or payment success.
- Gateway protects routes and forwards requests correctly.
- Dockerfiles and Kubernetes YAML files are available and tested.
- Word document, README, source code, and demo flow are ready.

8. Conclusion

The project can be achieved in two days only if all members work in parallel with clearly assigned responsibilities and fixed integration checkpoints. Success comes from disciplined coordination, not from reducing core functionality.